sltoo – Integrating Requirements into CI/CD

Kristoffer Nordström

September 5, 2021

info@sltoo.dev

sltoo – Integrating Requirements into CI/CD

Kristoffer Nordström
September 5, 2021
info@sltoo.dev

Motivation

Motivation

- It's an honour to open this topic's session.
- hen talking about requirements, we should also define the requirements of the problem we're trying to solve.
- Storytime

Motivation

- Fully automated traceability matrix
- Consistent and up-to-date documents from the source
- Store (requirements') meta-information with code

Motivation
- Fully automated traceability matrix
- Consistent and up-to-date documents from the source
- Store (requirements') meta-information with code

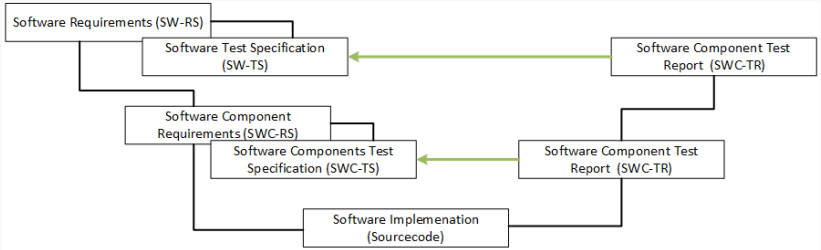
- It's fine manually. Requirements change find minimal delta-test? Change more than once. Hence we want this fully automated.
- Review multiple documents for a different contractor. No consistent set of documents were provided
- Jon Holt's term in introduction to MBSE: Documents are a live *view* of the system (not pretty pictures). CI/CD for software, why not for documents?
- It came with one document that contained all requirements and use-cases. It appeared to have been exported from some web-based tool. All the information therein is lost
- Also: Too technical, interface with management/business side
- 4'

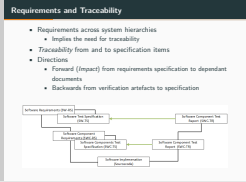
Theory

Theory

Requirements and Traceability

- Requirements across system hierarchies
  - Implies the need for traceability
- *Traceability* from and to specification items
- Directions
  - Forward (*Impact*) from requirements specification to dependant documents
  - Backwards from verification artefacts to specification





Here Traceability only from items/issues

Traceability

- *Requirement A*
  - Red button to shut down system
- *Implementation a* says implemented A
  - Traceability can be automated
  - Machine-readable
- What if *A* changes?
  - *A* knows nothing of *a*

Traceability
Requirement A
Red button to shut down system
Implementation a says implemented A
Traceability can be automated
Machine-readable
What if A changes?
A knows nothing of a
PANIC

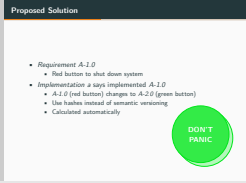
Traceability

- *Requirement A*
  - Green button with large friendly letters: don't panic
- *Implementation a* says implemented `A`
  - Traceability can be automated
  - Machine-readable
- What if *A* changes?
  - *A* knows nothing of *a*
  - Traceability isn't given anymore

Traceability
Requirement A
Green button with large friendly letters: don't panic
Implementation a says implemented A
Traceability can be automated
Machine-readable
What if A changes?
A knows nothing of a
Traceability isn't given anymore
DON'T PANIC

Proposed Solution

- *Requirement A-1.0*
  - Red button to shut down system
- *Implementation a* says implemented *A-1.0*
  - *A-1.0* (red button) changes to *A-2.0* (green button)
  - Use hashes instead of semantic versioning
  - Calculated automatically



Why hashes: no tool or manual changes required, it's all derived

Let's see how it looks on an example

Example Requirement

VCD Writer Inputs SW-AS-501
The output from *AggregatorInterface* can be fed into the *VCD Writer* to create an identical output file.
Rationale: Make the process as transparent as possible.
Depends on: SW-AS-500

Id:	SW-AS-501	Priority:	0.0	Owner:	development
Invented on:	2020-05-30	Invented by:	default	Status:	passed
Class:	detailable	Hash:	f8d68d11		

Hash is calculated over *Name*, *Description* and *Verification Method*.
Rationale is only informative

```
$ sha256sum "${Name}${Description}${VerifMethod}"
```

Example from pymergevcd's architecture specification

A from previous slides is now SW-AS-501

Version n.0 is now F8D68D11

Testing the Example Requirement

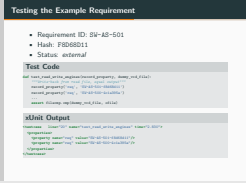
- Requirement ID: `SW-AS-501`
- Hash: `F8D68D11`
- Status: *external*

Test Code

```python
def test_read_write_engines(record_property, dummy_vcd_file):
    """Write-back from read file, equal output"""
    record_property('req', 'SW-AS-501-f8d68d11')
    record_property('req', 'SW-AS-500-4c1a395a')
    ...
    assert filecmp.cmp(dummy_vcd_file, ofile)
```

xUnit Output

```xml
<testcase   line="20" name="test_read_write_engines" time="2.830">
  <properties>
    <property name="req" value="SW-AS-501-f8d68d11"/>
    <property name="req" value="SW-AS-500-4c1a395a"/>
  </properties>
</testcase>
```

Traceability Matrix

Table A.1: Traceability Matrix Table

Req. ID	Status	UT
SW-AS-1	passed	passed
SW-AS-100	open	open
SW-AS-101	open	open
SW-AS-102	open	open
SW-AS-200	failed	failed
SW-AS-201	passed	passed
SW-AS-202	passed	passed
SW-AS-300	passed	passed
SW-AS-301	open	open
SW-AS-302	open	open
SW-AS-500	passed	passed
SW-AS-501	passed	passed

Now it should be straightforward to integrate it into any CI pipeline

Integrating Requirements into CI/CD

- Integration for every output document
- Match *open* and/or *failed* issues
  - Left as an excercise for the reader
- Example for *failed* issues

```
$ bash -ec 'test "$(grep -c failed \
    arch/artifacts/tracematrix.tex)" -eq "0"'
```


Integrating Requirements into CI/CD
- Integration for every output document
- Match *open* and/or *failed* issues
  - Left as an excercise for the reader
- Example for *failed* issues

$ bash -ec 'test "$(grep -c failed \
arch/artifacts/tracematrix.tex)" -eq "0"'

9'

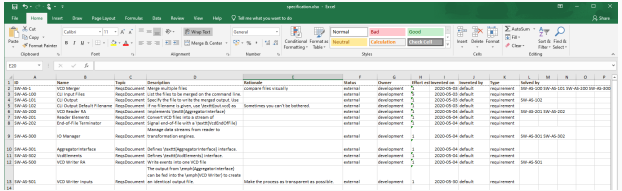
sltoo in Practice

sltoo in Practice

- Tracking requirements in text files with git –> might be ideal for engineering department
- Defining system behaviour is a team effort
- Solution not for everyone / clunky UI
- Guess: most people aren't familiar with the tooling/console –> Which tool is "everyone" familiar with?

Excel Workflow (I)

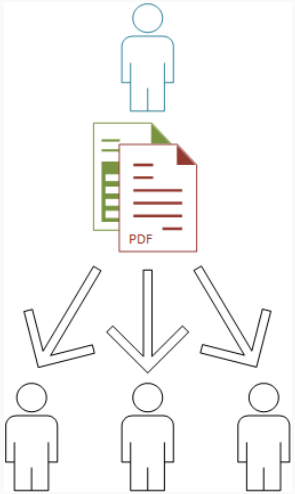
- Defining system collaborative effort
- Familiarity / Ease-of-use
- Consistency of Documents
  - The *Truth* is always in your repository
  - Templating for branding
- Works if all you've got is Office and E-Mail
- Getting Started: Edit example Excel-Sheet



Excel Workflow (I)
- Defining system collaborative effort
- Familiarity / Ease-of-use
- Consistency of Documents
  - The Truth is always in your repository
  - Templating for branding
- Works if all you've got is Office and E-Mail
- Getting Started: Edit example Excel-Sheet

Simply use the excel sheet as long as no documents or verification tasks are necessary

Excel Workflow (II) – Distribution



Excel Workflow (II) – Distribution

Document Baseline

Every document has a its own version tag

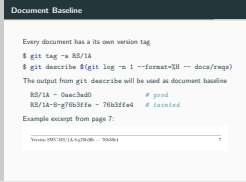
```
$ git tag -a RS/1A
$ git describe $(git log -n 1 --format=%H -- docs/reqs)
```

The output from `git describe` will be used as document baseline

```
RS/1A - 0aec3ad0              # good
RS/1A-8-g76b3ffe - 76b3ffe4   # tainted
```

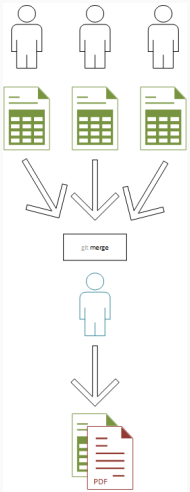
Example excerpt from page 7:

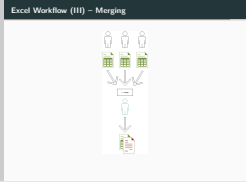
Version SWC-RS/1A-8-g76b3ffe — 76b3ffe4 7



Exercise to reject tainted documents in referenced documents

Excel Workflow (III) – Merging





Commit for every author individually -> also put name in commit for easier lookup

Conclusion

Conclusion

Storytime Revisited

- Requirements shipped with code ✓
  - Including relational meta-information
- Traceability matrix automated ✓
- Continuously updated documentation ✓
  - Document Versioning (baselining) ✓

A familiar UI for all stakeholders included

Storytime Revisited
- Requirements shipped with code ✓
  - Including relational meta-information
- Traceability matrix automated ✓
- Continuously updated documentation ✓
  - Document Versioning (baselining) ✓

A familiar UI for all stakeholders included

Your Jira workspace will be gone

Questions

Questions

Appendix

rmtoo – Introductions

An introduction presentation into *rmtoo* and with more details.

rmtoo – Introductions
An introduction presentation into *rmtoo* and with more details.

Traceability Rationale

- Traceability for the given requirements
- Bring code and documentation into same repository
- Integrate into build-system
  - Detect upstream changes to requirements
  - Quickly identify affected code-regions
- No silver bullet for verification

Traceability Rationale
- Traceability for the given requirements
- Bring code and documentation into same repository
- Integrate into build-system
  - Detect upstream changes to requirements
  - Quickly identify affected code-regions
- No silver bullet for verification

Results

The status *external* will yield the following results:

- *open*
  - No matching requirement ID
- *passed*
  - Matching requirement ID
  - All hashes match
  - Unit-tests passed
- *failed*
  - Matching requirement ID
  - Some/all hashes didn't match, or
  - Unit-tests haven't passed

Results

The status *external* will yield the following results:

- *open*
  - No matching requirement ID
- *passed*
  - Matching requirement ID
  - All hashes match
  - Unit-tests passed
- *failed*
  - Matching requirement ID
  - Some/all hashes didn't match, or
  - Unit-tests haven't passed

Installation

Traceability features are in the beta releases.

```
$ pip3 install sltoo>=25.1.0b3
$ wget https://kown7.github.io/pymergevcd/assets/template_p
```


Installation
Traceability features are in the beta releases.
$ pip3 install sltoo>=25.1.0b3
$ wget https://kown7.github.io/pymergevcd/assets/template_p

Alternatives

- Sphinx-Needs
- Octane ALM
- Codebeamer
- Aligned elements
- See Wikipedia
- ...

Alternatives
- Sphinx-Needs
- Octane ALM
- Codebeamer
- Aligned elements
- See Wikipedia
- ...

Future Developements

- Write Parser for *Test Reports* ✓
- Documents with the correct identifier automatically solve the specification
  - Document Formats:
    - docx (maybe with pandoc)
    - LaTeX✓
    - Text
- GUI with multi-documents support (RS/TS/..)
  - Simplify design process

Future Developements
Write Parser for Test Reports ✓
Documents with the correct identifier automatically solve the specification
Document Formats:
docx (maybe with pandoc)
LaTeX✓
Text
GUI with multi-documents support (RS/TS/..)
Simplify design process

Final Thoughts

- Never test against your requirements
- Always write some form of test specification

Final Thoughts
- Never test against your requirements
- Always write some form of test specification

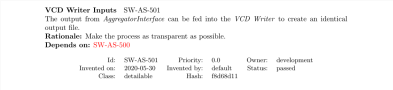
Licensing

This work is licensed under a Creative Commons "Attribution-NonCommercial-ShareAlike 3.0 Unported" license.

Licensing
This work is licensed under a Creative Commons "Attribution-NonCommercial-ShareAlike 3.0 Unported" license.

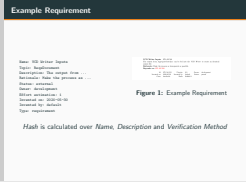
Example Requirement

```
Name: VCD Writer Inputs
Topic: ReqsDocument
Description: The output from ...
Rationale: Make the process as ...
Status: external
Owner: development
Effort estimation: 1
Invented on: 2020-05-30
Invented by: default
Type: requirement
```



VCD Writer Inputs SW-AS-501
The output from *AggregatorInterface* can be fed into the *VCD Writer* to create an identical output file.
Rationale: Make the process as transparent as possible.
Depends on: SW-AS-500

Id:	SW-AS-501	Priority:	0.0	Owner:	development
Invented on:	2020-05-30	Invented by:	default	Status:	passed
Class:	detailable	Hash:	f8d0fdd11		

Figure 1: Example Requirement

Hash is calculated over *Name*, *Description* and *Verification Method*